JavaScript Master Notes

This is a complete revision guide for the JavaScript files in this workspace.

---

1. Variables and Datatypes

const

Used for values that should not be reassigned.

```
const accountID = 13554523;
```

let

Used for variables whose value may change later.

```
let accountEmail = "surya@google.com";
```

var

Old way of declaring variables; avoid using it in modern JS because of scope issues.

```
var accountPassword = "1234";
```

undefined

A variable exists but has no assigned value.

```
let accountState;
console.log(accountState); // undefined
```

null

Represents an intentional empty value.

```
let temp = null;
```

Primitive Data Types

- String
- Number
- Boolean
- Null
- Undefined
- Symbol
- BigInt

Non-Primitive Data Types

- Object
- Array
- Function

`typeof`

```
console.log(typeof null); // object
console.log(typeof undefined); // undefined
```

> *JavaScript has a known quirk:* `typeof null` *returns* `"object"`.

---

2. Conversion and Type Coercion

Number to String

```
let score = 33;
let scoreInString = String(score);
```

String to Number

```
let score2 = "33abc";
let score2InNumber = Number(score2);
console.log(score2InNumber); // NaN
```

`NaN` means “Not a Number.”

Boolean conversion

```
let police = "surya";
let policeIsLoggedIn = Boolean(police);
console.log(policeIsLoggedIn); // true
```

Truthy values:

- non-empty strings
- non-zero numbers
- arrays, objects, functions

Falsy values:

- `0`
- `""`
- `null`
- `undefined`
- `false`
- `NaN`

---

3. Operators and Comparisons

Arithmetic

```
let value = 3;
let negValue = -value;
console.log(negValue); // -3
```

```
console.log(2 % 3); // 2
```

String concatenation

```
console.log("2" + 2 + 2); // "222"
```

Increment operator

```
let gameCounter = 100;
++gameCounter;
console.log(gameCounter); // 101
```

Comparison operators

```
console.log(2 > 1); // true
```

== vs ===

```
console.log("2" === 2); // false
```

Use === because it checks both value and type.

Null and undefined comparison quirks

```
console.log(null > 0);   // false
console.log(null == 0);  // false
console.log(null >= 0);  // true
console.log(undefined == 0); // false
```

These are strange JavaScript behaviors, so be careful.

---

4. Strings

Concatenation

```
let name = "Suryansh";
let repoCount = 2;

console.log("My name is: " + name + ", repo count is: " + repoCount);
```

Template literals

```
console.log(`Hello my name is ${name}, My Repo Count is ${repoCount}`);
```

String methods

```
const nameObj = new String("SURYANSH");
console.log(nameObj[3]);
console.log(nameObj.length);
console.log(nameObj.toUpperCase());
console.log(nameObj.charAt(0));
console.log(nameObj.substring(1, 4));
console.log(nameObj.slice(-7, -4));
```

trim()

```
const text = "   surya   ";
console.log(text.trim());
```

replace()

```
const url = "https://google.com/suryansh%20sahu";
console.log(url.replace('%20', '-'));
```

includes()

```
console.log(url.includes('google')); // true
```

split()

```
const nameString = "s-u-r-y-a";
console.log(nameString.split('-'));
```

---

5. Numbers and Math

Number object

```
const Balance = new Number(10000000);
console.log(Balance.toString());
console.log(Balance.toFixed(2));
```

toPrecision()

```
const number = 100.440134;
console.log(number.toPrecision(5));
```

toLocaleString('en-IN')

```
const num = 3257014;
console.log(num.toLocaleString('en-IN'));
```

Common Math methods

```
console.log(Math.abs(-4));
console.log(Math.round(4.2));
console.log(Math.ceil(4.2));
console.log(Math.floor(4.5));
console.log(Math.min(3, 6, 2, 4));
console.log(Math.max(3, 6, 2, 4));
```

Random numbers

```
console.log(Math.random());
console.log(Math.floor(Math.random() * 10) + 1);
```

Random range

```
const min = 10;
const max = 20;
console.log(Math.floor(Math.random() * (max - min + 1)) + min);
```

---

6. Date and Time

Current date

```
let myTime = new Date();
console.log(myTime);
```

Formatting

```
console.log(myTime.toString());
console.log(myTime.toDateString());
console.log(myTime.toLocaleString());
```

Custom dates

```
const myCreatedDate = new Date(2006, 6, 15, 12, 0);
console.log(myCreatedDate.toDateString());
```

Timestamp

```
let myDOB = new Date("2006-06-15");
let myTimestamp = Date.now();
console.log(myTimestamp);
console.log(myDOB.getTime());
```

---

7. Arrays

Creating arrays

```
const myArr = [0, 1, 2, 3, 4, 5];
const superheros = ["hanuman", "thor", "ironman"];
```

Accessing values

```
console.log(myArr[0]);
```

Array methods

```
myArr.push(6);
myArr.pop();
myArr.unshift(9);
myArr.shift();
```

join()

```
const newArr = myArr.join();
```

concat()

```
const allHeros = marvelHeros.concat(dcHeros);
```

Spread operator

```
const allNewHeros = [...marvelHeros, ...dcHeros];
```

Flatten nested arrays

```
const anotherArray = [1, 2, 3, [4, 5, 6, [7, 8, 9]]];
const realAnotherArray = anotherArray.flat(Infinity);
```

Array.isArray()

```
console.log(Array.isArray("Suryansh"));
```

Array.from()

```
console.log(Array.from("Suryansh"));
```

Array.of()

```
console.log(Array.of(100, 200, 300));
```

---

8. Objects

Object literal

```
const JsUser = {
  name: "Suryansh",
  age: 21,
  email: "surya@gmail.com",
  isLoggedin: true
};
```

Access values

```
console.log(JsUser.name);
console.log(JsUser["email"]);
```

Symbols as keys

```
const mySym = Symbol("key1");
const JsUser = {
  [mySym]: "myKey",
  name: "Suryansh"
};
```

Methods in object

```
JsUser.greeting = function () {
  console.log("Hello Js User");
};
```

Nested object

```
const regularUser = {
  email: "some@gmail.com",
  fullname: {
    firstname: "ram",
    lastname: "shyam"
  }
};
```

Merging objects

```
const obj1 = { 1: "a", 2: "b" };
const obj2 = { 3: "a", 4: "b" };
const obj3 = { ...obj1, ...obj2 };
```

Destructuring

```
const course = {
  coursename: "javascript",
  price: "$99",
  courseInstructor: "john"
};

const { courseInstructor: instructor, price: p } = course;
```

---

9. Functions

Basic function

```
function sayMyName() {
  console.log("Suryansh");
}
```

Return value

```
function addTwoNumbers(number1, number2) {
  return number1 + number2;
}
```

Default parameters

```
function loginUserMessage(username = "Suryansh") {
  return `${username} just logged in`;
}
```

---

10. Scope and Closures

Block scope

```
if (true) {
  let a = 10;
  const b = 20;
}
```

Nested scope

```
function one() {
  const username = "suryansh";

  function two() {
    console.log(username);
  }

  two();
}
```

Closure

```
function makeFunc() {
  const name = "Mozilla";

  function displayName() {
    console.log(name);
  }

  return displayName;
}
```

---

11. Arrow Functions

```
const addThree = (num1, num2, num3) => num1 + num2 + num3;
console.log(addThree(3, 4, 5));
```

Arrow functions do not have their own `this` ; they use lexical scope.

---

12. IIFE

Immediately Invoked Function Expressions run as soon as they are defined.

```
(function chai() {
  console.log("Hemlo Ji");
})();
```

---

13. Control Flow

If statement

```
if (isUserLoggedIn) {
  console.log("User is logged in");
}
```

If-Else

```
if (balance < 500) {
  console.log("less than 500");
} else if (balance < 750) {
  console.log("less than 750");
} else {
  console.log("greater than 750");
}
```

Logical operators

```
if (userLoggedIn && debitCard) {
  console.log("Allow to buy course");
}
```

```
if (loggedInFromGoogle || loggedInFromEmail) {
  console.log("User logged in");
}
```

Truthy / Falsy

```
const userEmail = [];
if (userEmail) {
  console.log("Got user email");
}
```

Switch

```
let month = 7;

switch (month) {
  case 1:
    console.log("January");
    break;
  default:
    console.log("Not January");
}
```

Ternary operator

```
const iceTeaPrice = 100;
iceTeaPrice <= 80 ? console.log("less than 80") : console.log("more than 80");
```

Nullish coalescing

```
let val1 = null ?? 10 ?? 20;
console.log(val1); // 10
```

---

14. Loops and Iteration

`for` loop

```
for (let i = 0; i <= 10; i++) {
  console.log(i);
}
```

while loop

```
let index = 0;
while (index <= 10) {
  console.log(index);
  index += 2;
}
```

do while

```
let score = 11;
do {
  console.log(score);
  score++;
} while (score <= 10);
```

for...of

```
const myArray = [2, 3, 4, 5];
for (const num of myArray) {
  console.log(num);
}
```

for...in

```
const myObj = { js: "Javascript", py: "Python" };
for (const key in myObj) {
  console.log(`${myObj[key]} shortcut is : ${key}`);
}
```

forEach()

```
const coding = ["Javascript", "C++", "Python"];

coding.forEach((val) => {
  console.log(val);
});
```

filter()

```
const myNums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const newNums = myNums.filter((num) => num > 4);
```

map()

```
const newNums = myNumbers.map((number) => number * 10);
```

reduce()

```
const total = myNums.reduce((acc, currVal) => acc + currVal, 0);
```

---

15. DOM (Document Object Model)

Select elements

```
const title = document.getElementById("title");
const firstItem = document.querySelector(".list_item");
const allItems = document.querySelectorAll(".list_item");
```

Change text and content

```
secondLang.innerHTML = "Mojo";
newli.textContent = "Gojo";
```

Create new elements

```
const div = document.createElement("div");
div.className = "main";
div.style.backgroundColor = "green";
document.body.appendChild(div);
```

Remove elements

```
const lastLang = document.querySelector("li:last-child");
lastLang.remove();
```

Events

```
buttons.forEach(function (button) {
  button.addEventListener("click", function (e) {
    console.log(e.target.id);
  });
});
```

---

16. Events

Click event

```
const gfg = document.querySelector('.gfg');
gfg.addEventListener('click', e);
```

Prevent default form behavior

```
form.addEventListener('submit', function (e) {
  e.preventDefault();
});
```

Keyboard event

```
window.addEventListener('keydown', (e) => {
  console.log(e.key, e.keyCode, e.code);
});
```

---

17. Promises, Async/Await, and Fetch

Promise

```
const promiseOne = new Promise(function (resolve, reject) {
  setTimeout(function () {
    resolve();
  }, 1000);
});
```

`then()` and `catch()`

```
promiseOne
  .then(() => console.log("Resolved"))
  .catch(() => console.log("Rejected"));
```

Async / Await

```
async function getData() {
  const response = await fetch("https://api.github.com/users/hiteshchoudhary");
  const data = await response.json();
  console.log(data);
}
```

Fetch API

```
fetch('https://api.github.com/users/hiteshchoudhary')
  .then((response) => response.json())
  .then((data) => console.log(data));
```

---

18. OOP and Classes

Constructor function

```
function User(username, loginCount, signedIn) {
  this.username = username;
  this.loginCount = loginCount;
  this.signedIn = signedIn;
}
```

`new` keyword

```
const userOne = new User("Suryansh", 15, true);
```

Class syntax

```
class User {
  constructor(username, email, password) {
    this.username = username;
    this.email = email;
    this.password = password;
  }

  changeUsername() {
    return this.username.toUpperCase();
  }
}
```

Inheritance

```
class Teacher extends User {
  constructor(username, email, password) {
    super(username);
    this.email = email;
    this.password = password;
  }
}
```

Getter and setter

```
class User {
  get email() {
    return this._email.toUpperCase();
  }

  set email(value) {
    this._email = value;
  }
}
```

Static method

```
class User {
  static createId() {
    return "123";
  }
}
```

---

19. Important JavaScript Concepts to Remember

- Prefer `let` and `const` over `var`
- Use `===` instead of `==`
- Use template literals for strings
- Understand truthy/falsy values
- Learn primitive vs reference data types
- Understand `this`, closure, and scope
- Use `map`, `filter`, `reduce` for array logic
- Use `addEventListener` for browser interaction
- Use `fetch` and `async/await` for API calls
- Learn objects, classes, and prototypes

---

20. Final Summary

This JavaScript course covers the core essentials required to become comfortable with web development:

- basics of JS
- arrays and objects
- functions and scope
- control flow and loops
- DOM and events
- async programming
- classes and OOP

If you keep revising these concepts and practice them by writing small examples, you will become confident in JavaScript very quickly.